

项目进展汇报

2026 年 6 月

总体方向

过去几周，AGENT调研了 161 篇关于自演化 agent、agent 记忆系统和工作流优化的论文，并深入分析了 ECC (github.com/affaan-m/ECC, 218k 星) ——目前最有影响力的开源自演化 coding agent 框架。

核心结论是定位调整：**项目的贡献不是 agent 框架本身，而是一套面向微纳光学的、可验证、可复现的论文复现系统。** Agent 基础设施只是手段。

之前两个问题的回答

这套系统相比一个聪明人直接用 Claude Code，真实增益在哪？

第一，物理验证自动化。 每次复现结束后自动检查能量守恒、光学定理、Rayleigh 极限、截断收敛等物理不变量。Claude Code 不具备这些概念。

第二，结构化溯源。 每次运行产出 attempt capsule 和 run manifest，记录使用了哪些 skill、检索了哪些记忆、用了什么参数和模型版本。结果可归因。Claude Code 不留这些痕迹。

第三，回放回归测试。 skill 更新后，自动检查已解决的旧任务是否退化。Claude Code 在习惯改变时静默丢失旧能力，无感知。

第四，跨 session 经验积累。 三层经验系统（提示词备注、skill、向量记忆库）跨论文、跨 session 传递经验。Claude Code 每次 session 从零开始。

第五，项目级状态树。 整个系统状态（skill、蓝图、记忆库、workflow 配置）在每次变更时快照，可以回滚到任意历史状态，也可以对比不同的论文处理顺序。Claude Code 没有等价机制。

固定场景的 agent 方案相比写死脚本，本质区别是什么？

9 步 workflow 中，agent 只出现在需要判断的环节：论文理解、物理建模、代码生成、理论检查、结果归因、复现性自检、报告撰写。两个环节是纯脚本：物理验证（数学不变量，纯计算）和导出（taint 检查、白名单、manifest，纯规则）。

如果某个环节在实践中证明足够确定，就降级为脚本。DSL 的价值只在“流程结构因论文而异”时才成立——而确实如此，不同论文需要不同的物理问题、不同的求解器、不同的验证策略。

与 ECC 的关系

ECC 以 218k 星和 10 个月的日常使用验证了“经验自动提取 + skill 聚类”这条路线是可行的。我们从 ECC 借鉴了 15 条具体的工程实践，包括按频次赋初始置信度、审查时只取最近 N 条记录（tail 采样）、四裁决质量门控（保存 / 改进后保存 / 合并到已有 / 丢弃）、自循环过滤、以及带超时看门狗的密钥脱敏正则等。

但 ECC 的范式与科研可复现性根本不兼容：

- **在线自演化与执行耦合。** 写代码的 agent 和提取经验的 agent 是同一个，在同一个 session 里。
- **LLM 判断是唯一的验证手段。** 没有确定性检查，没有回放，没有人工门控。
- **直接变异持久状态。** 没有沙箱，没有回滚，没有状态树。

这些特性在日常编码场景下可以接受（go build 和 go test 就是验证器），但在 AI4S 场景下不行——物理正确性和同行评审级别的可复现性是硬性要求。

核心设计决策：Worker 与 Evolver 严格分离

整个系统最关键的架构选择是**把生产 worker 和元认知 evolver 分离到独立的 Magnus 容器中，在执行和反思之间强制严格的只读边界。**

生产侧 (Worker)：

- 以 Magnus job 形式在隔离容器中运行
- 执行 9 步复现 workflow
- 产出（COMSOL capsule、代码、报告、验证结果）一旦生成即不可变
- 不能修改 skill、蓝图、workflow 拓扑或项目规则

反思侧 (Evolver)：

- 以独立的 Magnus job 运行，在积累 N 篇（如 10 篇）未审查的 run 后触发
- 以只读数据的方式读取 worker 产出——不作为可执行指令
- 提出 skill 候选、提示词备注修改建议、禁区建议
- 所有提议必须经过回放回归、安全检查和人工确认，才能修改任何持久状态
- 不允许迭代自身；其 workflow 拓扑和指令人工编写且固定

为什么这很重要：

这种分离确保科学输出保持确定性和可审计性。每一个复现结果都可以追溯到特定的系统状态，附带完整的溯源信息。审稿人可以复现任意一次运行的精确条件。这个性质是统一 CLI agent（如 ECC）的"黑箱状态变异"无法满足的。

正如我们在分析中指出的：

通用编码 agent（如 218k 星的 ECC）证明了在线自演化与执行合一的可行性，但这种范式与垂域科学可复现性根本不兼容。在我们的场景中，生产轨迹（COMSOL 作业）必须是临时的、不可变的，而元认知演化必须是离线的、有门控的。因此，我们将 Worker 和 Evolver 分离到独立的 Magnus 容器中，在执行与反思之间强制严格的只读边界。这确保科学输出（COMSOL capsule）保持确定性和可审计性，满足同行评审期刊的可复现性要求——这是统一 CLI agent 的"黑箱状态变异"无法调和的性质。

系统架构

Project-Flow（最上层）

类 git 的项目级状态树。每个节点是系统状态的完整快照：CLI 配置、工作 workflow 全套（YAML、skill、蓝图、记忆库）、自迭代 workflow 全套（独立的一套）、历史上下文（已审查/未审查的运行列表）、人工介入记录。

状态变更只有三种方式：论文复现、自迭代、人工介入。每次变更生成新的不可变节点，旧节点永不修改。支持 checkout、diff、回滚、跨分支对比、从历史状态重跑。

回传白名单（纯 Python 脚本，不是 agent）拒绝任何超出允许范围的修改并记录风险警告。例如，如果 worker 试图修改 workflow YAML 或自迭代 skill，修改被拒绝，尝试被记录。

存储三地同步：本地磁盘（日常开发）、Gustation（大文件、COMSOL 产出）、GitHub 私有仓库（仅文本部分的云端备份）。

工作 Workflow（9 步）

- 1. PDF 预处理——文字/图表/公式提取，曲线图特殊处理
- 2. 论文阅读——理解、外部检索、记忆检索、识别论文本身疏漏
- 3. 复现目标设计——拆分目标、定义依赖关系和验收标准
- 4. 理论推导与代码实现——解析推导和数值脚本编写
- 5. 理论检查——带反模式清单的对抗式审查，判断不符是我们错了还是论文错了
- 6. 运行与监视——提交 Magnus 作业，监视收敛性和稳定性
- 7. 物理通用检查——agent 选择适用的检查项，脚本执行（能量守恒、光学定理、极限趋势、截断收敛、因果对称性、数值合理性）
- 8. 结果分析——与论文对比，结合物理检查结论归因
- 9. 复现性自检——重跑、参数扰动、网格收敛，排除巧合
- 10. 总结与报告——经验提取、记忆写入、自迭代报告、人类复现报告（区分 pipeline / job / physical）

在隔离的 Magnus 容器中运行，产出临时且不可变。

自迭代 Workflow（5 步，积累约 10 篇后触发）

- 1. 并发审查——10 个子 agent 各审一篇，审查 capsule/报告/产出，发起质询
- 2. 聚类与规划——识别跨论文的重复模式和物理技巧，规划 skill 修改
- 3. 并发 skill 工作——每个 skill 一个子 agent，产出候选
- 4. 验证与回放——格式检查、回放回归测试、安全检查
- 5. 报告与人工确认——治理报告，人工确认后才能修改持久状态

在独立的 Magnus 容器中运行，以只读方式读取 worker 产出，不允许迭代自身。

三层经验系统

层级	规模	治理方式	注入方式
提示词备注	每节点不超过 3 条	轻量生命周期，AI 添加，人工可见	追加到指令后
Skill + 蓝图	几十到几百条	证据门控（2+ case 或人工确认）；蓝图从属于 skill	按节点白名单加载
向量记忆库	几十万条	程序化治理：写入侧过滤、定期合并、衰减、冲突消解	向量召回 → store routing → 8B reranker → top-5

硬件：qdrant + Qwen3-Embedding-0.6B + Qwen3-Reranker-8B（INT8 量化，常驻 RTX 5880 Ada，约占 8GB 显存）。微纳光学仿真很少用 GPU，reranker 与 COMSOL 作业共存无资源争抢。

当前风险状态

通过 161 篇论文调研和 ECC 分析，所有风险已识别，风险清单无遗漏。

- **P0 (5 项)**：设计层全部解决，尚未实现。
- **P1 (8 项)**：3 项设计层解决，5 项有详细方案但尚未实现。
- **P2 (6 项)**：建议实施。
- **P3 (2 项)**：远期。

客观评估：设计已经比较完整，但还没有写代码，也没有端到端跑通任何一个 case。

下一步

1. 查阅 5-10 篇提示词工程论文（1 天，限时完成）
2. 编写 `energy_consensation.py` 和最小版 `workflow.py`（本地跑通一个 Mie case）
3. 构建 Docker 镜像（[workflow.py](#) + opencode CLI + MCP server）
4. 集成 Magnus 作业提交
5. 实现最小版 project-flow（snapshot、checkout、log）
6. 开始在真实论文上实践，积累经验

优先级很明确：**先端到端跑通一个 case，再逐步补充治理基础设施。**